Day 109: Gradient Boosting From Scratch — Real Sequential Error Correction

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 5 | Complete study guide

Watch: Gradient Boost Part 1 (of 4): Regression Main Ideas (StatQuest) | Gradient Boost Part 2 (of 4): Regression Details (StatQuest)

Script: `day109_gradient_boosting.py` · every number in this guide is copied by program from `day109_run_output.txt`, the real output of that script.

Gradient boosting written from scratch, for squared error and for log loss. Each tree fits what the model before it got wrong, and on Diabetes training error falls in every round from 79.29 to 5.62, while test error reaches 57.13 and then climbs to 63.77. Where it stands against the alternatives is mixed and useful: it beats a forest on Titanic (+0.0177), loses to one on breast cancer (-0.0111), and loses to plain linear regression on Diabetes (58.07 against 54.77).

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

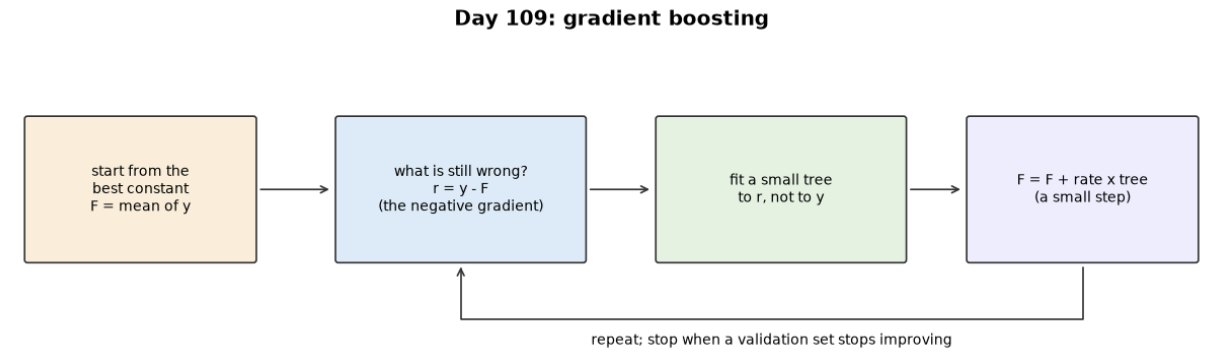
Boosting builds a model in sequence: each new small model is trained on what the current model still gets wrong, and a fraction of its answer is added.

It is called **gradient** boosting because 'what is still wrong' is the negative gradient of the loss. For squared error, that is simply the **residual**, y – F. Day 107's forest reduces variance by averaging independent trees; boosting mainly reduces bias by adding dependent ones.

REAL-WORLD SCENARIO

Correcting an essay in rounds: the first reader fixes the worst problems, the next reader works only on what remains, and each changes a little, so that no reader's mistakes take over.

SYSTEM DIAGRAM



Gradient boosting: start from a constant, then repeatedly fit a small tree to what is still wrong.

Today's part for Day 111

`day109_exercise.py` is the regression tree and the boosting loop for both losses. Its real output:

ACTUAL OUTPUT

```
mean of y = 3.333; residuals of rows 5, 6 = 2.667, 4.667; the rest average -1.833
one stump, rate 0.5: left step -0.917, right step +1.833; predictions [2.417, 2.417, 2.417, 2.417,
    5.167, 5.167]
zero rounds gives the best constant: [3.333, 3.333, 3.333, 3.333, 3.333, 3.333]
rate 1, deep enough to isolate rows: training error 0.0e+00
log loss, 50 rounds: probabilities [0.0, 0.0, 0.0, 0.0, 1.0, 1.0] for labels [0, 0, 0, 0, 1, 1]
predict_boost equals the tracked training prediction: largest gap 0.0e+00
```

Data: Diabetes (442 patients, 10 measurements, a disease-progression score) and breast cancer, both bundled with scikit-learn, and Titanic. Boosting is written in NumPy; scikit-learn is run only as an outside check. Section 5 uses a synthetic exclusive-or on purpose.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: three rounds on six numbers

DEFINITION — start here, no prior knowledge assumed

Start from the mean. Compute each row's residual. Fit a tree with one question (a **stump**) to the residuals, and add half of its answer. Repeat.

REAL-WORLD SCENARIO

Estimating a bill: start with the average, then adjust for the biggest item, then for what is left.

MATHEMATICS

$$F_0 = \text{mean of } y, \quad r_m = y - F_{m-1}$$

$$F_m = F_{m-1} + \nu\, h_m(x), \quad h_m = \text{a small tree fitted to } r_m, \quad \nu = \text{learning rate}$$

IMPLEMENTATION

```
def fit_boost(X, y, loss, rounds, lr, max_depth, evals=(), min_leaf=1):
    y = y.astype(float); idx = np.arange(len(y))
    base = y.mean() if loss == "squared" else np.log(y.mean() / (1 - y.mean()))
    F = np.full(len(y), base); trees = []
    E = [np.zeros((rounds + 1, len(v))) + base for v in evals]
    for m in range(rounds):
        if loss == "squared":
            r = y - F
            leaf = lambda i: r[i].mean()
        else:
            p = sigmoid(F); r = y - p; h = p * (1 - p)
            leaf = lambda i: r[i].sum() / max(h[i].sum(), 1e-12)
        t = grow_reg(X, r, idx, max_depth, min_leaf); set_leaves(t, leaf); trees.append(t)
        F = F + lr * predict_reg(t, X)
        for e, v in zip(E, evals):
            e[m + 1] = e[m] + lr * predict_reg(t, v)
    return {"base": base, "lr": lr, "trees": trees, "train": F}, E
```

ACTUAL OUTPUT

```
y = [1.0, 1.0, 2.0, 2.0, 6.0, 8.0]; start from the mean, 3.333; learning rate 0.5, stumps (depth 1)
round 1: residuals [-2.333, -2.333, -1.333, -1.333, 2.667, 4.667]; split x <= 4.5; leaves -1.833 / +3.667
         prediction [2.417, 2.417, 2.417, 2.417, 5.167, 5.167]; squared error 13.083
round 2: residuals [-1.417, -1.417, -0.417, -0.417, 0.833, 2.833]; split x <= 4.5; leaves -0.917 / +1.833
         prediction [1.958, 1.958, 1.958, 1.958, 6.083, 6.083]; squared error 5.521
round 3: residuals [-0.958, -0.958, 0.042, 0.042, -0.083, 1.917]; split x <= 5.5; leaves -0.383 / +1.917
         prediction [1.767, 1.767, 1.767, 1.767, 5.892, 7.042]; squared error 2.215
Actual output from this lesson's run.
```

What it means

Check round 1: the mean is 3.333, the two large rows have residuals 2.667 and 4.667, the stump's right leaf is their average (+3.667), and half of that is added. Squared error falls 13.083, 5.521, 2.215. In round 3 the question changes to $x \le 5.5$: once the first problem is partly fixed, the largest remaining error is the last row alone. That shift of attention is sequential error correction.

WHY CHOOSE THIS?

Each step is simple enough to do by hand.

WHY NOT THIS? (TRADEOFFS)

Each tree depends on all the earlier ones, so the training cannot run in parallel the way a forest's can.

2. Correctness checks

DEFINITION — start here, no prior knowledge assumed

For log loss, the model predicts log-odds F, and each row's residual is y – p. Leaves take one **Newton step**: the sum of gradients over the sum of curvatures. That approximates the exact best step, which can be found by a search.

REAL-WORLD SCENARIO

A map gives the direction (the gradient); the curve of the road (the curvature) says how far to go in one step.

MATHEMATICS

$$L = -y \log p - (1 - y) \log(1 - p), \quad p = 1 / (1 + e^{-F})$$

$$-dL/dF = y - p, \quad d^2L/dF^2 = p(1 - p)$$

$$\text{leaf value} = \Sigma(y - p) / \Sigma\, p(1 - p)$$

ACTUAL OUTPUT

```
30 random problems: the fast split search finds the brute-force reduction in squared error in 30 of 30
log loss: numerical gradient against -(y - p): largest gap 3.7e-10
a real leaf: one Newton step 0.5126; the exact best step by grid search 0.5240 (Newton is an approximation of
    it)
training log loss fell in 200 of 200 rounds, from 0.6599 to 0.0001
diabetes, 5 splits, 100 rounds, same settings as scikit-learn: test RMSE gap +0.0929 on average; largest
    prediction gap 14.9336 (target std 77.0; tied splits may differ)
Actual output from this lesson's run.
```

What it means

The split search agrees with brute force in 30 of 30. The gradient matches a numerical one to 3.7e-10. On a real leaf, Newton gives 0.5126 and the exact best step is 0.5240: close, and an approximation, as the formula says. Training log loss fell in 200 of 200 rounds. Against scikit-learn with the same settings the test RMSE gap is +0.0929, although single predictions can differ by up to 14.9336. Tied splits are the likely cause, though that is not tested here; Day 110 runs the test against XGBoost.

WHY CHOOSE THIS?

Each piece is checked separately, including against a library.

WHY NOT THIS? (TRADEOFFS)

An average match hides individual differences, which is why the largest gap is printed too.

3. Sequential error correction on real data

DEFINITION — start here, no prior knowledge assumed

On Diabetes, a depth-3 tree and a learning rate of 0.1. 'corr' asks whether each tree really is fitting the residuals it was given.

REAL-WORLD SCENARIO

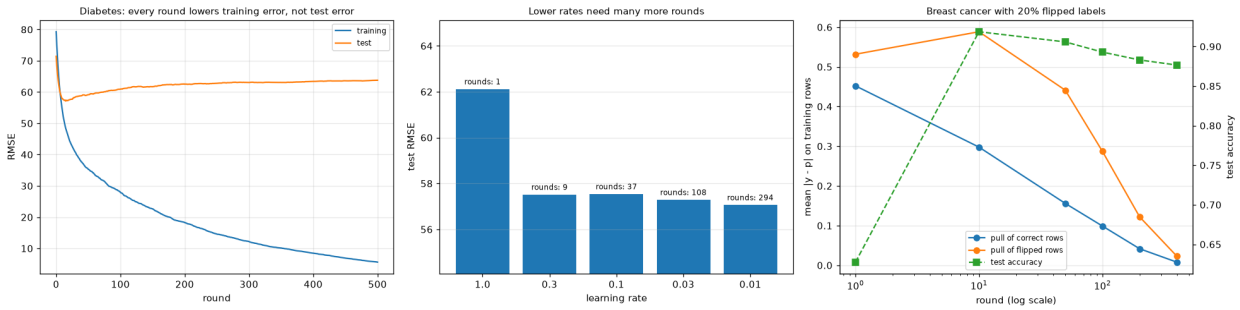
A tutor who keeps setting only the questions a student got wrong: marks on those questions rise, even when the student is memorising.

SPECIFICATION

Data	442 patients; 309 training, 133 test; the raw measurements.
Model	Depth 3, learning rate 0.1, 500 rounds, squared error.
Note	The lowest test row is printed for this table only. Section 4 chooses the round on validation rows.

ACTUAL OUTPUT

```
309 training, 133 test patients; depth 3; learning rate 0.1; the mean alone gives test RMSE 71.42
'corr' is the correlation between that round's tree and the residuals it was trained on
  round  train RMSE  test RMSE   corr  worst residual
      0       79.29      71.42    nan           193.9
      1       74.84      68.31  0.758           182.5
      2       71.07      65.71  0.719           171.1
      5       62.27      61.05  0.630           151.5
     10       53.55      57.86  0.520           127.6
     25       42.59      57.70  0.332           108.1
     50       35.29      59.18  0.289            95.0
    100       27.90      60.93  0.325            79.0
    200       18.25      62.52  0.279            64.0
    500        5.62      63.77  0.277            28.3
lowest test RMSE 57.13 at round 15 (read off the test set, so for this table only); at round 500: 63.77, while
    training reaches 5.62
Actual output from this lesson's run.
```



Left: Diabetes, training and test error. Middle: learning rates, with the rounds validation chose. Right: 20% flipped labels: the pull of each kind of row, and test accuracy.

What it means

The correction is real: each tree correlates with its residuals (0.758 at first), and the worst training error shrinks from 193.9 to 28.3. But the correlation falls to about 0.28: later trees explain less and less of what is left. Training reaches 5.62, while test error turns around at round 15 and climbs to 63.77. Boosting, unlike a forest, *does* overfit with more trees.

WHY CHOOSE THIS?

Very strong on training error, with a simple loop.

WHY NOT THIS? (TRADEOFFS)

Needs a stopping rule, because more rounds eventually hurt.

4. Learning rate against rounds

DEFINITION — start here, no prior knowledge assumed

Shrinkage adds only a fraction of each tree. A lower rate needs more rounds. The round is chosen on a validation part, and the test part is used once.

REAL-WORLD SCENARIO

Walking down a hill in fog: small steps are slower, and you are less likely to overshoot.

IMPLEMENTATION

```
a, te = split_rows(len(y), 0.2, s); tr, va = a[len(a) // 4:], a[:len(a) // 4]
_, E = fit_boost(X[tr], y[tr], "squared", MAX_ROUNDS, lr, DEPTH, evals=(X[va], X[te]))
b = stop_round(E[0], y[va], rmse); rounds.append(b); scores.append(rmse(E[1][b], y[te])); edge +=
    b == MAX_ROUNDS
```

ACTUAL OUTPUT

```
10 splits: 60% training, 20% validation (chooses the round), 20% test (used once); depth 3; at
    most 1500 rounds
   rate  mean rounds  at the limit  test RMSE     std
    1.0          1.2             0      62.12    4.51
    0.3          8.9             0      57.52    3.01
    0.1         37.2             0      57.54    3.75
   0.03        108.5             0      57.29    3.64
   0.01        293.8             0      57.07    3.61
Actual output from this lesson's run.
```

What it means

A rate of 1 is clearly worse (62.12), and stops after 1.2 rounds on average. Below that, rates from 0.3 to 0.01 are indistinguishable (57.52 to 57.07, with a spread of about 3.6 between splits), and cost 8.9 to 293.8 rounds. The common advice to use a small rate is sound here, but the benefit is small and the cost is large. The limit of 1,500 rounds was never reached, so the grid is not cut off.

WHY CHOOSE THIS?

Shrinkage makes the choice of rounds much less delicate.

WHY NOT THIS? (TRADEOFFS)

Lower rates cost proportionally more time, for a gain too small to see here.

5. Tree depth: what stumps can and cannot do

DEFINITION — start here, no prior knowledge assumed

A stump asks about one feature, so boosted stumps add up one-feature effects: an **additive** model. Depth d allows interactions of up to d features. An exclusive-or is pure interaction, with no effect from either feature alone.

REAL-WORLD SCENARIO

Two medicines that are harmless alone and dangerous together: no rule about one medicine can find it.

ACTUAL OUTPUT

```
10 splits; learning rate 0.1; at most 500 rounds; the round chosen on validation rows
 depth  diabetes RMSE  rounds  xor accuracy  rounds
     1          56.35   188.4        0.5020     3.4
     2          56.43    47.0        0.8989   346.8
     3          57.54    37.2        0.9236   168.3
     4          57.95    21.7        0.9238   101.8
     6          61.94    19.0        0.9128    49.8
     8          63.92     9.8        0.8861    36.0
the exclusive-or is synthetic on purpose; two perfect questions would reach about 0.95
Actual output from this lesson's run.
```

What it means

Two opposite results. On Diabetes stumps are best (56.35) and deep trees worst (63.92), which suggests that the signal is mostly additive, and section 8 agrees. On the exclusive-or, stumps cannot learn at all (0.5020: a coin toss); depth 2 jumps to 0.8989 and depth 3 reaches 0.9236, well beyond the roughly 0.80 of Day 106's single pruned tree on the same kind of data (a different experiment, so only a rough comparison). Many small steps appear to get around the horizon effect, provided each tree can see the interaction.

WHY CHOOSE THIS?

Depth directly sets which interactions are possible.

WHY NOT THIS? (TRADEOFFS)

Deeper trees overfit sooner, so depth 3 to 6 is the usual range.

6. Bias and variance against Day 107's forest

DEFINITION — start here, no prior knowledge assumed

Day 107's measurement, repeated on the same 30 training sets. The forest is rebuilt here so that both are in one table.

REAL-WORLD SCENARIO

One team reduces disagreement between estimators; the other keeps refining a single estimate.

ACTUAL OUTPUT

```
Titanic: Day 107's 30 training sets of 300 passengers and the same 268 test passengers; learning rate 0.1,
    depth 3
model                     variance  bias+noise   Brier  accuracy
forest (64 trees)           0.0181      0.1351  0.1532    0.7868
boosting, 1 round           0.0004      0.2168  0.2172    0.6157
boosting, 5 rounds          0.0020      0.1694  0.1714    0.8021
boosting, 25 rounds         0.0067      0.1348  0.1415    0.7948
boosting, 100 rounds        0.0171      0.1336  0.1506    0.7975
boosting, 400 rounds        0.0439      0.1404  0.1843    0.7710
Actual output from this lesson's run.
```

What it means

Boosting moves the opposite way to a forest. As rounds are added, bias+noise falls (0.2168 to 0.1336) while variance grows (0.0004 to 0.0439). The best total is in the middle: 0.1415 at 25 rounds, beating the forest's 0.1532; at 400 rounds boosting has lost it (0.1843). This is the classic trade-off, measured: a forest starts from low-bias trees and averages away variance, and boosting starts from a high-bias constant and adds complexity until variance costs more.

WHY CHOOSE THIS?

Explains why the two methods need different settings.

WHY NOT THIS? (TRADEOFFS)

Thirty overlapping subsets of one table: the pattern is more reliable than the exact numbers.

7. Wrong labels: when correcting errors goes wrong

DEFINITION — start here, no prior knowledge assumed

Boosting works hardest on the rows it gets wrong. If a row is wrong because its *label* is wrong, that effort is wasted on noise. Here 20% of training labels are flipped and test labels stay correct, as on Day 106.

REAL-WORLD SCENARIO

A teacher who keeps drilling a student on the questions with mistakes in the answer key.

ACTUAL OUTPUT

```
breast cancer, 20% of training labels flipped, test labels correct, 10 splits; boosting at learning rate 0.1,
    depth 3
'pull' is the mean size of the residual |y - p|: how hard the next tree is pushed by those rows
  round  test acc  pull: correct rows  pull: flipped rows
      1    0.6275              0.4521              0.5318
     10    0.9187              0.2978              0.5884
     50    0.9058              0.1559              0.4411
    100    0.8930              0.0988              0.2869
    200    0.8830              0.0416              0.1219
    400    0.8766              0.0079              0.0229
early stopping on a noisy validation part: test 0.9047, after 28.1 rounds; Day 107's forest on the same noisy
    labels: 0.9333
Actual output from this lesson's run.
```

What it means

At round 10, flipped rows pull about twice as hard as correct ones (0.5884 against 0.2978), and test accuracy is at its best (0.9187). The model then steadily memorises them: their pull falls to 0.0229 because the model has learned the wrong answers, and test accuracy drops to 0.8766. Early stopping recovers part of it (0.9047), but Day 107's forest does better on the same labels (0.9333). Averaging ignores noisy rows; sequential correction seeks them out.

WHY CHOOSE THIS?

Early stopping is a cheap partial defence.

WHY NOT THIS? (TRADEOFFS)

With noisy labels a forest is the safer default.

8. Is it better? Against trees, forests and a straight line

DEFINITION — start here, no prior knowledge assumed

The same 20 splits per dataset. On Diabetes the honest baseline is **linear regression**, one weight per feature, which has no trees at all.

REAL-WORLD SCENARIO

Before buying a sports car for the school run, check how a bicycle does.

ACTUAL OUTPUT

```
20 splits; boosting uses learning rate 0.1, depth 3, at most 500 rounds, stopped on 25% of the training part;
    forest 100 trees
Titanic: pruned tree 0.8093, forest 0.7996, boosting 0.8174
  boosting minus forest +0.0177 [+0.0090, +0.0263]; better on 16, worse on 2 of 20
Breast cancer: pruned tree 0.9243, forest 0.9596, boosting 0.9485
  boosting minus forest -0.0111 [-0.0187, -0.0044]; better on 4, worse on 13 of 20
Diabetes (RMSE, lower is better): the mean 75.55, boosting 58.07, linear regression 54.77
  boosting minus linear regression +3.30 [+2.09, +4.41]; boosting better on 2 of 20
the splits share rows, so these intervals are too narrow, a rough guide only
Actual output from this lesson's run.
```

What it means

No method wins everywhere. Titanic: boosting 0.8174, ahead of both the pruned tree and a default forest. Breast cancer: the forest (0.9596) beats boosting (0.9485), on 13 of 20 splits. Diabetes: a straight line beats boosting by +3.30 RMSE and boosting wins only 2 of 20 splits. With 442 patients and a signal that section 5 suggests is mostly additive, the flexible model has more to lose than to gain. Always run the simple baseline.

WHY CHOOSE THIS?

Often the best method on tabular data, once tuned and stopped.

WHY NOT THIS? (TRADEOFFS)

More settings than a forest, more sensitive to noise, and sometimes beaten by a straight line.

Interview preparation — technical questions

Q: What does each tree fit?

A: The negative gradient of the loss at the current model: the residual y - F for squared error, y - p for log loss.

Q: Why is it called gradient boosting?

A: It is gradient descent in function space: each tree approximates one step down the loss.

Q: What is the leaf value for log loss?

A: One Newton step: the sum of gradients over the sum of p(1 - p). Here 0.5126 against an exact 0.5240.

Q: How do you stop it?

A: Watch a validation set and keep the best round; here round 15 of 500 on Diabetes, read from the test set for illustration only.

Q: Which matters more, learning rate or rounds?

A: They trade off. Rates 0.3 to 0.01 were indistinguishable once validation chose the rounds.

Interview preparation — theoretical questions

Q: Bagging or boosting?

A: Bagging averages independent strong trees and cuts variance. Boosting adds dependent weak trees and cuts bias.

Q: Why do more trees hurt boosting but not forests?

A: A forest averages, so variance has a floor. Boosting keeps fitting: 0.0004 to 0.0439 variance here.

Q: Why are shallow trees used?

A: Depth limits interactions and each step's size; depth 1 was best on Diabetes, but cannot learn an exclusive-or (0.5020).

Q: Why is boosting sensitive to label noise?

A: It pushes hardest on rows it gets wrong: flipped rows pull about twice as hard, then get memorised.

Key takeaway and what's next

KEY TAKEAWAY

Gradient boosting is a short loop: fit a small tree to what is still wrong, and add a little of it. It lowers bias steadily, and that same persistence lets it fit noise, so it needs early stopping. Measured against forests and a straight line, it wins some and loses some; the method to beat depends on the data, which is why a simple baseline always comes first.

- Diabetes: training 79.29 to 5.62; test 57.13 at round 15, then 63.77 at round 500.
- Learning rates 0.3 to 0.01: 57.07 to 57.52, at 8.9 to 293.8 rounds.

- Bias+noise 0.2168 to 0.1336 while variance grows 0.0004 to 0.0439; best Brier 0.1415 at 25 rounds, forest 0.1532.
- 20% flipped labels: best 0.9187, then 0.8766; early stopping 0.9047; forest 0.9333.
- Diabetes: linear regression 54.77, boosting 58.07.

Suggested watch order, if starting from zero

- 1. Gradient Boost Part 1 (of 4): Regression Main Ideas (StatQuest) — the main idea on a small table.
- 2. Gradient Boost Part 2 (of 4): Regression Details (StatQuest) — the algorithm, step by step.

Day 110 preview

- **XGBoost.** A production boosting library: is it the same algorithm, and what does it add? Compared prediction by prediction with a from-scratch version.